

Algorytmy Zaawansowane

Przykładowe instance do powtórki

Po trzy instance na algorytm — same dane wejściowe, do samodzielnego prześledzenia.

Spis treści

1	Para najbliższych punktów	2
2	Mnożenie ciągu macierzy (Matrix-Chain-Order)	2
3	Najdłuższy wspólny podciąg (LCS)	2
4	Zachłanny wybór zajęć	3
5	Algorytm Huffmana	3
6	Szeregowanie zadań z deadlineami	4
7	Greedy-Set-Cover	4
8	Approx-Vertex-Cover	5
9	Exact-Subset-Sum	5
10	Approx-Subset-Sum	6
11	Maksymalne skojarzenie w grafie dwudzielnym (MM)	6
A	Rozwiązania	8
A.1	Para najbliższych punktów – instancja 1	8
A.2	Matrix-Chain-Order – instancja 1	9
A.3	Najdłuższy wspólny podciąg (LCS) – instancja 1	11
A.4	Zachłanny wybór zajęć – instancja 1	12
A.5	Algorytm Huffmana – instancja 1	12
A.6	Szeregowanie zadań z deadlineami – instancja 1	13
A.7	Greedy Set Cover – instancja 1	14
A.8	Approx Vertex Cover – instancja 1	14
A.9	Exact Subset Sum – instancja 1	15
A.10	Approx Subset Sum – instancja 1	15
A.11	Maximum Matching – instancja 1	17
A.12	Para najbliższych punktów – instancja 2	18
A.13	Matrix-Chain-Order – instancja 2	19
A.14	Najdłuższy wspólny podciąg (LCS) – instancja 2	21
A.15	Zachłanny wybór zajęć – instancja 2	22
A.16	Algorytm Huffmana – instancja 2	22
A.17	Szeregowanie zadań z deadlineami – instancja 2	24
A.18	Greedy Set Cover – instancja 2	24
A.19	Approx Vertex Cover – instancja 2	25
A.20	Exact Subset Sum – instancja 2	25
A.21	Approx Subset Sum – instancja 2	26
A.22	Maximum Matching – instancja 2	27

Para najbliższych punktów

Przykład 1.1 (Najbliższe punkty – instancja 1). $Q = \{(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4), (8, 9), (1, 1)\}.$

Znaleźć parę punktów w najmniejszej odległości (euklidesowej).

Przykład 1.2 (Najbliższe punkty – instancja 2).

Znaleźć parę punktów w najmniejszej odległości:

p_i	1	2	3	4	5	6	7	8
x	0	1	4	5	7	8	9	11
y	0	6	2	5	1	7	3	6

Przykład 1.3 (Najbliższe punkty – instancja 3). $Q = \{(1, 2), (4, 4), (7, 1), (8, 8), (2, 9), (10, 3), (6, 5), (3, 3)\}.$

Znaleźć parę punktów w najmniejszej odległości (euklidesowej).

Mnożenie ciągu macierzy (Matrix-Chain-Order)

Przykład 2.1 (Matrix-Chain-Order – instancja 1). $n = 4, \quad p = (5, 4, 6, 2, 7),$ czyli $A_1 (5 \times 4), A_2 (4 \times 6), A_3 (6 \times 2), A_4 (2 \times 7).$ Wyznaczyć tablice m i s oraz optymalne nawiasowanie.**Przykład 2.2 (Matrix-Chain-Order – instancja 2).** $n = 5, \quad p = (10, 3, 12, 5, 50, 6),$ czyli $A_1 (10 \times 3), A_2 (3 \times 12), A_3 (12 \times 5), A_4 (5 \times 50), A_5 (50 \times 6).$ Wyznaczyć tablice m i s oraz optymalne nawiasowanie.**Przykład 2.3 (Matrix-Chain-Order – instancja 3).** $n = 5, \quad p = (4, 10, 3, 12, 20, 7),$ czyli $A_1 (4 \times 10), A_2 (10 \times 3), A_3 (3 \times 12), A_4 (12 \times 20), A_5 (20 \times 7).$ Wyznaczyć tablice m i s oraz optymalne nawiasowanie.

Najdłuższy wspólny podciąg (LCS)

Przykład 3.1 (LCS – instancja 1). $X = (A, B, C, B, D, A, B), \quad Y = (B, D, C, A, B, A).$

Wyznaczyć tablicę długości LCS oraz jeden najdłuższy wspólny podciąg.

Przykład 3.2 (LCS – instancja 2). $X = (G, A, T, T, A, C, A), \quad Y = (G, C, A, T, A, A, T).$

Wyznaczyć tablicę długości LCS oraz jeden najdłuższy wspólny podciąg.

Przykład 3.3 (LCS – instancja 3).

$X = (A, B, A, C, B, D, A)$, $Y = (B, A, B, C, D, A, B)$.

Wyznaczyć tablicę długości LCS oraz jeden najdłuższy wspólny podciąg.

Zachłanny wybór zajęć

Przykład 4.1 (Wybór zajęć – instancja 1).

Znaleźć podzbiór o największej liczności parami zgodnych zajęć:

i	1	2	3	4	5	6	7
s_i	1	3	0	5	3	5	8
t_i	4	5	6	7	9	9	10

Przykład 4.2 (Wybór zajęć – instancja 2).

Znaleźć podzbiór o największej liczności parami zgodnych zajęć:

i	1	2	3	4	5	6	7	8	9
s_i	0	2	1	6	4	8	7	9	11
t_i	3	5	4	8	7	9	11	12	14

Przykład 4.3 (Wybór zajęć – instancja 3).

Znaleźć podzbiór o największej liczności parami zgodnych zajęć:

i	1	2	3	4	5	6	7	8
s_i	1	2	0	3	4	6	5	7
t_i	3	5	7	8	9	10	9	12

Algorytm Huffmana

Przykład 5.1 (Huffman – instancja 1).

Znaki i ich częstości:

znak	a	b	c	d	e	f
częstość	45	13	12	16	9	5

Skonstruować drzewo Huffmana i przypisać słowa kodowe.

Przykład 5.2 (Huffman – instancja 2).

Zaczynamy ze zbiorem wierzchołków: $(g, 2)$, $(o, 3)$, $(k, 6)$, $(m, 8)$, $(l, 11)$, $(u, 14)$, $(e, 20)$.

Skonstruować drzewo Huffmana i przypisać słowa kodowe.

Przykład 5.3 (Huffman – instancja 3).

Znaki i ich częstości:

znak	a	b	c	d	e	f	g
częstość	30	25	20	15	10	7	3

Skonstruować drzewo Huffmana i przypisać słowa kodowe.

Szeregowanie zadań z deadlinami

Przykład 6.1 (Szeregowanie zadań – instancja 1).

Zadania (kary w_i za spóźnienie, deadliny d_i):

i	1	2	3	4	5	6	7
d_i	4	2	4	3	1	4	6
w_i	100	90	70	60	50	30	15

Wyznaczyć harmonogram minimalizujący sumę kar zadań spóźnionych.

Przykład 6.2 (Szeregowanie zadań – instancja 2).

Wyznaczyć harmonogram minimalizujący sumę kar dla następujących zadań:

i	1	2	3	4	5	6	7
d_i	1	1	3	2	5	4	3
w_i	60	55	45	35	25	20	10

Przykład 6.3 (Szeregowanie zadań – instancja 3).

Zadania (kary w_i za spóźnienie, deadliny d_i); wyznaczyć harmonogram minimalizujący sumę kar:

i	1	2	3	4	5	6	7
d_i	3	1	2	4	2	1	5
w_i	40	50	30	25	20	35	10

Greedy-Set-Cover

Przykład 7.1 (Greedy Set Cover – instancja 1).

$X = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$,

$S_1 = \{1, 2, 3, 4, 5, 6\}$,

$S_2 = \{5, 6, 8, 9\}$,

$S_3 = \{1, 4, 7, 10\}$,

$S_4 = \{2, 5, 7, 8, 11\}$,

$S_5 = \{3, 6, 9, 12\}$,

$S_6 = \{10, 11\}$.

$\mathcal{F} = \{S_1, \dots, S_6\}$. Znaleźć pokrycie zwracane przez algorytm zachłanny.

Przykład 7.2 (Greedy Set Cover – instancja 2).

$X = \{1, \dots, 9\}$,

$S_1 = \{1, 2, 3, 4, 5\}$,

$S_2 = \{6, 7, 8, 9\}$,

$S_3 = \{1, 4, 7\}$,

$S_4 = \{2, 5, 8\}$,

$S_5 = \{3, 6, 9\}$.

$\mathcal{F} = \{S_1, \dots, S_5\}$. Znaleźć pokrycie zwracane przez algorytm zachłanny.

Przykład 7.3 (Greedy Set Cover – instancja 3).

$X = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$,

$S_1 = \{1, 2, 3, 4, 5\}$,

$S_2 = \{4, 5, 6, 7\}$,

$S_3 = \{1, 6, 8\}$,

$S_4 = \{2, 7, 9\}$,

$S_5 = \{3, 8, 9, 10\}$,

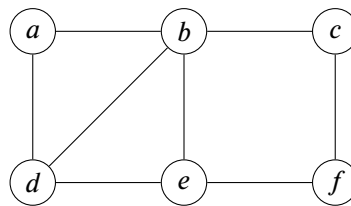
$S_6 = \{5, 10\}$.

$\mathcal{F} = \{S_1, \dots, S_6\}$. Znaleźć pokrycie zwracane przez algorytm zachłanny.

Approx-Vertex-Cover

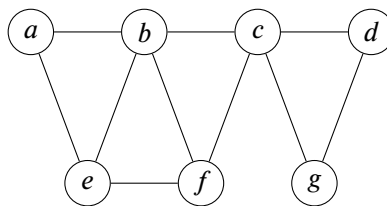
Przykład 8.1 (Approx Vertex Cover – instancja 1).

Wyznaczyć pokrycie wierzchołkowe zwracane przez Approx-Vertex-Cover dla grafu:



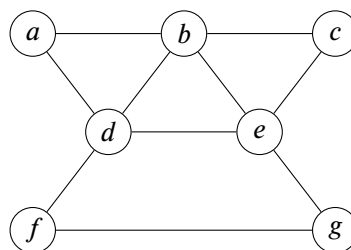
Przykład 8.2 (Approx Vertex Cover – instancja 2).

Wyznaczyć pokrycie wierzchołkowe zwracane przez Approx-Vertex-Cover dla grafu:



Przykład 8.3 (Approx Vertex Cover – instancja 3).

Wyznaczyć pokrycie wierzchołkowe zwracane przez Approx-Vertex-Cover dla grafu:



Exact-Subset-Sum

Przykład 9.1 (Exact Subset Sum – instancja 1).

$S = \{1, 4, 5, 11, 12\}$ (w kolejności $x_1, \dots, x_5$), $t = 21$.

Wyznaczyć kolejne listy L_i i największą sumę podzbioru $\leq t$.

Przykład 9.2 (Exact Subset Sum – instancja 2).

$S = \{2, 3, 7, 8, 10\}$ (w kolejności $x_1, \dots, x_5$), $t = 15$.

Wyznaczyć kolejne listy L_i i największą sumę podzbioru $\leq t$.

Przykład 9.3 (Exact Subset Sum – instancja 3).

$S = \{3, 5, 6, 9, 11\}$ (w kolejności $x_1, \dots, x_5$), $t = 20$.

Wyznaczyć kolejne listy L_i i największą sumę podzbioru $\leq t$.

Approx-Subset-Sum

Przykład 10.1 (Approx Subset Sum – instancja 1).

$S = \{106, 103, 205, 104\}$ (w kolejności $x_1, \dots, x_4$), $t = 312$, $\varepsilon = 0,4$.

Prześledzić działanie z przycinaniem ($\delta = \frac{\varepsilon}{2n}$): wyznaczyć listy L_i i zwracaną sumę.

Przykład 10.2 (Approx Subset Sum – instancja 2).

$S = \{10, 12, 17, 25\}$ (w kolejności $x_1, \dots, x_4$), $t = 50$, $\varepsilon = 0,5$.

Prześledzić działanie z przycinaniem ($\delta = \frac{\varepsilon}{2n}$): wyznaczyć listy L_i i zwracaną sumę.

Przykład 10.3 (Approx Subset Sum – instancja 3).

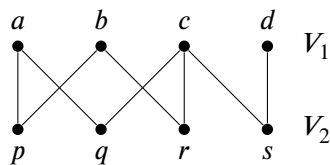
$S = \{8, 15, 23, 42\}$ (w kolejności $x_1, \dots, x_4$), $t = 70$, $\varepsilon = 0,3$.

Prześledzić działanie z przycinaniem ($\delta = \frac{\varepsilon}{2n}$): wyznaczyć listy L_i i zwracaną sumę.

Maksymalne skojarzenie w grafie dwudzielnym (MM)

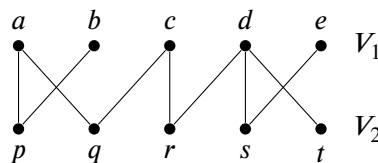
Przykład 11.1 (Maximum Matching – instancja 1).

Wyznaczyć najliczniejsze skojarzenie (ścieżki powiększające) w grafie:



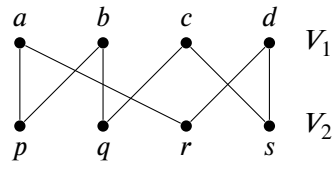
Przykład 11.2 (Maximum Matching – instancja 2).

Wyznaczyć najliczniejsze skojarzenie (ścieżki powiększające) w grafie:



Przykład 11.3 (Maximum Matching – instancja 3).

Wyznaczyć najliczniejsze skojarzenie (ścieżki powiększające) w grafie:



Rozwiązania

Para najbliższych punktów – instancja 1

Dane: $Q = \{(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4), (8, 9), (1, 1)\}$, $n = 8$. Ponumerujemy punkty w kolejności wejściowej:

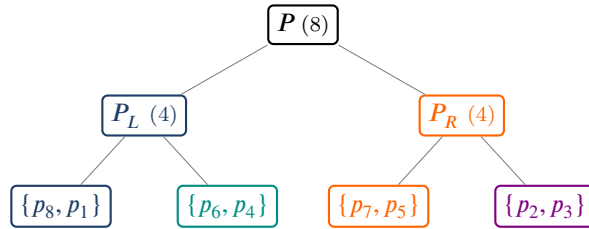
$$p_1 = (2, 3), p_2 = (12, 30), p_3 = (40, 50), p_4 = (5, 1), p_5 = (12, 10), p_6 = (3, 4), p_7 = (8, 9), p_8 = (1, 1).$$

Sortowanie wstępne. Tworzymy tablice X (rosnąco po x) i Y (rosnąco po y):

$$X = [p_8(1, 1), p_1(2, 3), p_6(3, 4), p_4(5, 1), p_7(8, 9), p_5(12, 10), p_2(12, 30), p_3(40, 50)],$$

$$Y = [p_8(1, 1), p_4(5, 1), p_1(2, 3), p_6(3, 4), p_7(8, 9), p_5(12, 10), p_2(12, 30), p_3(40, 50)].$$

Dziel. $|P| = 8 > 3$, dzielimy prostą $l: x = 5$ na $|P_L| = |P_R| = 4$, a każdą połowę dalej, aż do podzbiorów ≤ 3 punktów (liście — sprawdzane siłowo). Strukturę rekursji i kolory grup zbiera rysunek 1.



Rysunek 1: Drzewo rekursji. Cztery liście (po 2 punkty) liczone są siłowo; ich kolory powtarzają się w tekście i na rysunku 2.

$$P_L = \{p_8, p_1, p_6, p_4\}, \quad P_R = \{p_7, p_5, p_2, p_3\}.$$

Zwycięzaj — lewa połowa P_L . Liście (po 2 punkty — siłowo):

$$d(p_8, p_1) = \sqrt{1^2 + 2^2} = \sqrt{5} \approx 2,24, \quad d(p_6, p_4) = \sqrt{2^2 + 3^2} = \sqrt{13} \approx 3,61,$$

więc $\delta = \sqrt{5}$. Faza *połącz* w P_L : pas $x \in [2 - \sqrt{5}, 2 + \sqrt{5}]$; ponieważ $2 < \sqrt{5} < 3$, jego końce leżą w $(-1, 0)$ oraz $(4, 5)$, więc pas odrzuca $p_4(5, 1)$, zostaje $Y' = [p_8(1, 1), p_1(2, 3), p_6(3, 4)]$. Sprawdzamy:

$$d(p_8, p_1) = \sqrt{5}, \quad d(p_8, p_6) = \sqrt{13}, \quad d(p_1, p_6) = \sqrt{1^2 + 1^2} = \sqrt{2} \approx 1,41.$$

Zatem $\delta_L = \sqrt{2}$ dla pary $(p_1(2, 3), p_6(3, 4))$.

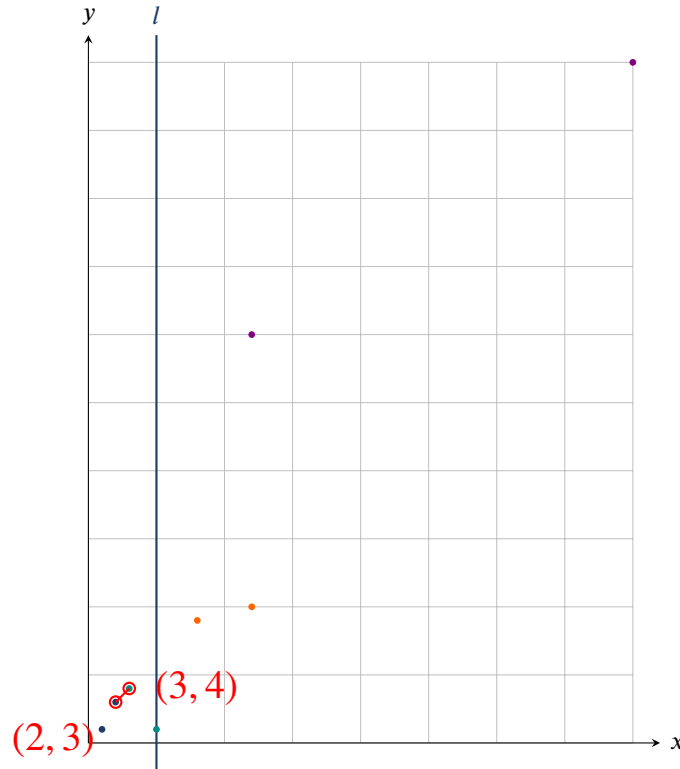
Zwycięzaj — prawa połowa P_R . Liście:

$$d(p_7, p_5) = \sqrt{4^2 + 1^2} = \sqrt{17} \approx 4,12, \quad d(p_2, p_3) = \sqrt{28^2 + 20^2} = \sqrt{1184} \approx 34,4,$$

więc $\delta = \sqrt{17}$. Faza *połącz*: ponieważ $4 < \sqrt{17} < 5$, pas $x \in [12 - \sqrt{17}, 12 + \sqrt{17}]$ ma końce w $(7, 8)$ oraz $(16, 17)$, więc $Y' = [p_7(8, 9), p_5(12, 10), p_2(12, 30)]$ (odpada p_3). Żadna para z Y' nie jest bliższa niż $\sqrt{17}$. Zatem $\delta_R = \sqrt{17}$ dla pary $(p_7(8, 9), p_5(12, 10))$.

Połącz — poziom górny. $\delta = \min(\delta_L, \delta_R) = \min(\sqrt{2}, \sqrt{17}) = \sqrt{2}$. Pas wokół $l: x = 5$ ma szerokość $2\delta = 2\sqrt{2}$; ponieważ $1 < \sqrt{2} < 2$, pas $x \in [5 - \sqrt{2}, 5 + \sqrt{2}]$ ma końce w $(3, 4)$ oraz $(6, 7)$. W pasie leży tylko $p_4(5, 1)$, więc $Y' = [p_4]$ — brak pary „międzystronowej”, δ bez zmian.

Wynik. Para najbliższych punktów to $(2, 3)$ i $(3, 4)$ w odległości $\delta = \sqrt{2} \approx 1,41$.



Rysunek 2: Instancja 1: prosta podziału $l: x = 5$, punkty pokolorowane wg liścia drzewa rekursji oraz znaleziona para $(2, 3), (3, 4)$ (czerwona).

Matrix-Chain-Order – instancja 1

Dane: $n = 4$, $p = (5, 4, 6, 2, 7)$, czyli

$$A_1 (5 \times 4), \quad A_2 (4 \times 6), \quad A_3 (6 \times 2), \quad A_4 (2 \times 7).$$

Wypełniamy tablice m i s *przekątnymi* po rosnącej długości podciągu L ; w każdym kroku dopisane komórki wyróżniono na **pomarańczowo**, a $\times$ to pola nieużywane ($i > j$ dla m , $i \geq j$ dla s). Skróót: $A_i = p_{i-1} \times p_i$.

Stan początkowy ($L = 1$, pojedyncze macierze, koszt 0):

$$m \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 \\ 1 & 0 & & & \\ 2 & \times & 0 & & \\ 3 & \times & \times & 0 & \\ 4 & \times & \times & \times & 0 \end{bmatrix}$$

$$s \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 \\ 1 & \times & & & \\ 2 & \times & \times & & \\ 3 & \times & \times & \times & \\ 4 & \times & \times & \times & \times \end{bmatrix}$$

i	0	1	2	3	4
p_i	5	4	6	2	7

 $A_i = p_{i-1} \times p_i$

$$L = 2 : \quad m(1, 2) = 5 \cdot 4 \cdot 6 = 120$$

$$m(2, 3) = 4 \cdot 6 \cdot 2 = 48$$

$$m(3, 4) = 6 \cdot 2 \cdot 7 = 84$$

po $L = 2$ (pary sąsiednich macierzy):

m	$i \backslash j$	1	2	3	4
	1	0	120		
	2	×	0	48	
	3	×	×	0	84
	4	×	×	×	0

s	$i \backslash j$	1	2	3	4
	1	×	1		
	2	×	×	2	
	3	×	×	×	3
	4	×	×	×	×

i	0	1	2	3	4
p_i	5	4	6	2	7

 $A_i = p_{i-1} \times p_i$

$$L = 3 : \quad m(1, 3) = \min \begin{cases} m(1, 1) + m(2, 3) + 5 \cdot 4 \cdot 2 \\ m(1, 2) + m(3, 3) + 5 \cdot 6 \cdot 2 \end{cases} = \min \begin{cases} 0 + 48 + 40 \\ 120 + 0 + 60 \end{cases} = \min \begin{cases} 88 \\ 180 \end{cases} = 88$$

$$m(2, 4) = \min \begin{cases} m(2, 2) + m(3, 4) + 4 \cdot 6 \cdot 7 \\ m(2, 3) + m(4, 4) + 4 \cdot 2 \cdot 7 \end{cases} = \min \begin{cases} 0 + 84 + 168 \\ 48 + 0 + 56 \end{cases} = \min \begin{cases} 252 \\ 104 \end{cases} = 104$$

po $L = 3$:

m	$i \backslash j$	1	2	3	4
	1	0	120	88	
	2	×	0	48	104
	3	×	×	0	84
	4	×	×	×	0

s	$i \backslash j$	1	2	3	4
	1	×	1	1	
	2	×	×	2	3
	3	×	×	×	3
	4	×	×	×	×

i	0	1	2	3	4
p_i	5	4	6	2	7

 $A_i = p_{i-1} \times p_i$

$$L = 4 : \quad m(1, 4) = \min \begin{cases} m(1, 1) + m(2, 4) + 5 \cdot 4 \cdot 7 \\ m(1, 2) + m(3, 4) + 5 \cdot 6 \cdot 7 \\ m(1, 3) + m(4, 4) + 5 \cdot 2 \cdot 7 \end{cases} = \min \begin{cases} 0 + 104 + 140 \\ 120 + 84 + 210 \\ 88 + 0 + 70 \end{cases} = \min \begin{cases} 244 \\ 414 \\ 158 \end{cases} = 158$$

tablice końcowe:

m	$i \backslash j$	1	2	3	4
	1	0	120	88	158
	2	×	0	48	104
	3	×	×	0	84
	4	×	×	×	0

s	$i \backslash j$	1	2	3	4
	1	×	1	1	3
	2	×	×	2	3
	3	×	×	×	3
	4	×	×	×	×

Optymalne nawiasowanie. Odtwarzamy je z tablicy s , rekurencyjnie dzieląc $A_i \dots A_j$ w miejscu $s(i, j)$:

$$s(1, 4) = 3 \Rightarrow (A_1 A_2 A_3)(A_4), \quad s(1, 3) = 1 \Rightarrow (A_1)(A_2 A_3).$$

Stąd optymalne nawiasowanie

$$(A_1 (A_2 A_3)) A_4$$

o minimalnym koszcie $m(1, 4) = 158$ mnożeń skalarnych.

Najdłuższy wspólny podciąg (LCS) – instancja 1

Dane: $X = (A, B, C, B, D, A, B)$, $Y = (B, D, C, A, B, A)$, czyli $m = 7$, $n = 6$. Tablicę kosztów c wypełniamy wierszami, korzystając z reguły

$$c(i, j) = \begin{cases} 0 & \text{jeśli } i = 0 \text{ lub } j = 0, \\ c(i-1, j-1) + 1 & \text{jeśli } i, j > 0 \text{ i } x_i = y_j, \\ \max\{c(i-1, j), c(i, j-1)\} & \text{jeśli } i, j > 0 \text{ i } x_i \neq y_j, \end{cases}$$

a równolegle zapisujemy w tablicy b kierunek, z którego wzięliśmy wartość ($\searrow$ – dopasowanie, $\uparrow$ – góra, $\leftarrow$ – lewo; remisy na korzyść góry). Komórki wchodzące do wybranego LCS wyróżniono na **pomarańczowo**.

Tablica kosztów c :

	j	0	1	2	3	4	5	6
i		y_j	B	D	C	A	B	A
0	x_i	0	0	0	0	0	0	0
1	A	0	0	0	0	1	1	1
2	B	0	1	1	1	1	2	2
3	C	0	1	1	2	2	2	2
4	B	0	1	1	2	2	3	3
5	D	0	1	2	2	2	3	3
6	A	0	1	2	2	3	3	4
7	B	0	1	2	2	3	4	4

Tablica strzałek b :

	j	0	1	2	3	4	5	6
i		y_j	B	D	C	A	B	A
0	x_i							
1	A		↑	↑	↑	↖	←	↖
2	B		↖	←	←	↑	↖	←
3	C		↑	↑	↖	←	↑	↑
4	B		↖	↑	↑	↑	↖	←
5	D		↑	↖	↑	↑	↑	↑
6	A		↑	↑	↑	↖	↑	↖
7	B		↖	↑	↑	↑	↖	↑

Dygresja. Po jednej komórce na każdą gałąź reguły (pozostałe 42 liczymy analogicznie), $X = (x_1, \dots, x_7)$, $Y = (y_1, \dots, y_6)$:

- **Dopasowanie** ($\searrow$): $c(2, 1)$, $x_2 = B = y_1$, więc $c(2, 1) = c(1, 0) + 1 = 1$.
- **Góra** ($\uparrow$): $c(3, 1)$, $x_3 = C \neq B = y_1$; sąsiedzi $c(2, 1) = 1 \geq c(3, 0) = 0$, więc $c(3, 1) = 1$.
- **Lewo** ($\leftarrow$): $c(2, 2)$, $x_2 = B \neq D = y_2$; sąsiedzi $c(1, 2) = 0 < c(2, 1) = 1$, więc $c(2, 2) = 1$.
- **Remis**: $c(1, 1)$, $A \neq B$, $c(0, 1) = c(1, 0) = 0$ – warunek $\geq$ spełniony, więc $\uparrow$.

Odtworzenie podciagu. Startujemy z $(m, n) = (7, 6)$ i idziemy za strzałkami b aż do brzegu; przy $\searrow$ schodzimy po przekątnej, dopisując x_i do LCS:

$$\begin{aligned} (7, 6) &\xrightarrow{\uparrow} (6, 6) \xrightarrow{\searrow} (5, 5) [x_6 = A] \xrightarrow{\uparrow} (4, 5) \xrightarrow{\searrow} (3, 4) [x_4 = B] \\ &\xrightarrow{\leftarrow} (3, 3) \xrightarrow{\searrow} (2, 2) [x_3 = C] \xrightarrow{\leftarrow} (2, 1) \xrightarrow{\searrow} (1, 0) [x_2 = B]. \end{aligned}$$

Znaki dopisywane są *po* powrocie z rekurencji, więc w kolejności od najgłębszego: B, C, B, A . Stąd najdłuższy wspólny podciąg

$$\boxed{\text{LCS}(X, Y) = (B, C, B, A)}$$

o długości $c(7, 6) = 4$. (Nie jest jedyny – np. (B, D, A, B) też ma długość 4.)

Zachłanny wybór zajęć – instancja 1

Dane ($n = 7$ zajęć), już posortowane po rosnącym czasie zakończenia t_i . Algorytm GREEDYACTIVITYSELECTOR wybiera z_1 , a następnie idzie po zajęciach rosnąco i dodaje z_i do rozwiązania A wtedy i tylko wtedy, gdy jest zgodne z ostatnio wybranym z_j , tj. $s_i \geq t_j$. Wybrane zajęcia wyróżniono na **pomarańczowo**.

i	1	2	3	4	5	6	7
s_i	1	3	0	5	3	5	8
t_i	4	5	6	7	9	9	10

Dygresja. Przebieg pętli (j – indeks ostatnio wybranego zajęcia, początkowo $A = \{z_1\}$, $j = 1$, $t_j = 4$):

- $i = 2$: $s_2 = 3 \geq t_j = 4$? *nie* – pomijamy.
- $i = 3$: $s_3 = 0 \geq 4$? *nie* – pomijamy.
- $i = 4$: $s_4 = 5 \geq 4$? *tak* – $A \cup \{z_4\}$, $j \leftarrow 4$, $t_j = 7$.
- $i = 5$: $s_5 = 3 \geq 7$? *nie* – pomijamy.
- $i = 6$: $s_6 = 5 \geq 7$? *nie* – pomijamy.
- $i = 7$: $s_7 = 8 \geq 7$? *tak* – $A \cup \{z_7\}$, $j \leftarrow 7$.

Wynik. Wybrane zajęcia są parami zgodne, bo ich przedziały $\langle 1, 4 \rangle$, $\langle 5, 7 \rangle$, $\langle 8, 10 \rangle$ są rozłączne:

$$A = \{z_1, z_4, z_7\}$$

o liczności $|A| = 3$.

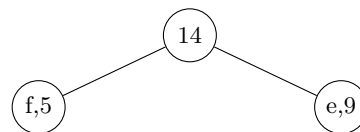
Algorytm Huffmana – instancja 1

Dane: $C = \{a, b, c, d, e, f\}$ z częstościami $f(a) = 45$, $f(b) = 13$, $f(c) = 12$, $f(d) = 16$, $f(e) = 9$, $f(f) = 5$. Algorytm Huffmana buduje drzewo *od dołu*: w każdej z $n - 1 = 5$ iteracji wyjmuje z kolejki dwa korzenie o *najmniejszej* częstości, łączy je nowym wierzchołkiem z o częstości $f(z) = f(x) + f(y)$ (mniejszy jako lewe dziecko) i wkłada z z powrotem. Krawędź do lewego dziecka etykietujemy 0, do prawego 1.

1. Łączymy f i e .

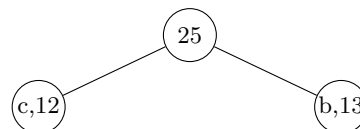
Zostają:

$(c, 12), (b, 13), (14), (d, 16), (a, 45)$.



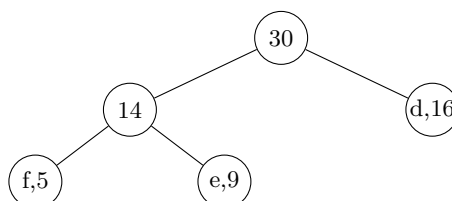
2. Łączymy c i b .

Zostają: $(14), (d, 16), (25), (a, 45)$.

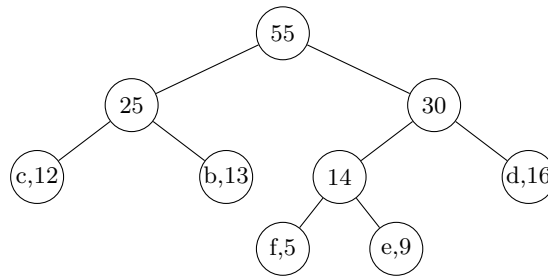


3. Łączymy 14 i d .

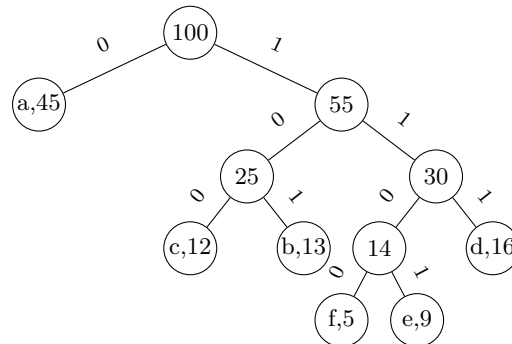
Zostają: $(25), (30), (a, 45)$.



4. Łączymy 25 i 30.
Zostają: $(a, 45), (55)$.



5. Łączymy a i 55.
(ostateczne drzewo kodów)



Odczytany kod z drzewa w kroku 5:

$a \Rightarrow 0$
 $c \Rightarrow 100$
 $b \Rightarrow 101$
 $d \Rightarrow 111$
 $f \Rightarrow 1100$
 $e \Rightarrow 1101$

Liczba bitów. Długość zakodowanego pliku to $B(T) = \sum_{c \in C} f(c) d_T(c)$:

$$B(T) = 45 \cdot 1 + 12 \cdot 3 + 13 \cdot 3 + 16 \cdot 3 + 5 \cdot 4 + 9 \cdot 4 = 224 \text{ bitów}$$

(wobec $58 \cdot 3 = 300$ bitów przy kodzie stałej długości po 3 bity na znak).

Szeregowanie zadań z deadlineami – instancja 1

Dane: $n = 7$ zadań o jednostkowym czasie wykonania, deadliney d_i i kary w_i . Rodzina zbiorów wolnych od kar tworzy matroid, więc optymalne rozwiązanie daje algorytm GREEDY: bierzemy zadania w kolejności *nierosnącej* kary i dodajemy z_i do A wtedy i tylko wtedy, gdy A pozostaje wolny od kar, tzn. profil $N_t(A) = |\{z_j \in A : d_j \leq t\}|$ nie przekracza $(1, 2, \dots, n)$. Zadania wybrane (terminowe) wyróżniono na **pomarańczowo**.

i	1	2	3	4	5	6	7
d_i	4	2	4	3	1	4	6
w_i	100	90	70	60	50	30	15

Dygresja. Kary są już posortowane nierosnąco ($w_1 \geq \dots \geq w_7$), więc zadania rozważamy w kolejności $z_1, \dots, z_7$. Poniżej A wypisane rosnąco po deadlineach, a po prawej profil $N(A) = (N_1, \dots, N_7)$, który musi pozostać $\leq (1, 2, 3, 4, 5, 6, 7)$:

- | | |
|--|--------------------------------|
| 1. $A = (z_1)$ | $N(A) = (0, 0, 0, 1, 1, 1, 1)$ |
| 2. $A = (z_2, z_1)$ | $N(A) = (0, 1, 1, 2, 2, 2, 2)$ |
| 3. $A = (z_2, z_1, z_3)$ | $N(A) = (0, 1, 1, 3, 3, 3, 3)$ |
| 4. $A = (z_2, z_4, z_1, z_3)$ | $N(A) = (0, 1, 2, 4, 4, 4, 4)$ |
| 5. $A = (z_2, z_4, z_1, z_3)$ (z_5 nie dodajemy, bo $N_4(A \cup \{z_5\}) = 5 > 4$) | |
| 6. $A = (z_2, z_4, z_1, z_3)$ (z_6 nie dodajemy, bo $N_4(A \cup \{z_6\}) = 5 > 4$) | |
| 7. $A = (z_2, z_4, z_1, z_3, z_7)$ | $N(A) = (0, 1, 2, 4, 4, 5, 5)$ |

Harmonogram. Zadania terminowe ustawiamy rosnąco po deadlinech, a spóźnione (z_5, z_6) na końcu:

$$(z_2, z_4, z_1, z_3, z_7, z_5, z_6)$$

Suma kar zadań spóźnionych: $w_5 + w_6 = 50 + 30 = 80$.

Greedy Set Cover – instancja 1

Dane: $X = \{1, \dots, 12\}$ oraz rodzina $\mathcal{F} = \{S_1, \dots, S_6\}$. Algorytm GSC startuje z $U = X$ i w każdej iteracji dokłada do pokrycia $\mathcal{C}$ ten zbiór $S \in \mathcal{F}$, który pokrywa najwięcej elementów jeszcze niepokrytych, tj. maksymalizuje $|S \cap U|$ (remisy rozstrzygamy na korzyść mniejszego indeksu), po czym odejmuje $U \leftarrow U \setminus S$. Zbiory wybrane do pokrycia wyróżniono na **pomarańczowo**.

S	elementy	$ S $
S_1	$\{1, 2, 3, 4, 5, 6\}$	6
S_2	$\{5, 6, 8, 9\}$	4
S_3	$\{1, 4, 7, 10\}$	4
S_4	$\{2, 5, 7, 8, 11\}$	5
S_5	$\{3, 6, 9, 12\}$	4
S_6	$\{10, 11\}$	2

Dygresja. W każdym wierszu po lewej bieżący zbiór niepokrytych U , a po prawej rozważane przecięcia $|S \cap U|$ (pogrubiony zwycięzca):

- | | |
|--|-------------|
| 1. $U = \{1, \dots, 12\}$: $ S_1 \cap U = \mathbf{6}$, $ S_4 \cap U = 5$, pozostałe ≤ 4 | wybór S_1 |
| 2. $U = \{7, 8, 9, 10, 11, 12\}$: $ S_4 \cap U = \mathbf{3}$, $ S_2 \cap U = S_3 \cap U = S_5 \cap U = S_6 \cap U = 2$ | wybór S_4 |
| 3. $U = \{9, 10, 12\}$: $ S_5 \cap U = \mathbf{2}$, $ S_2 \cap U = S_3 \cap U = S_6 \cap U = 1$ | wybór S_5 |
| 4. $U = \{10\}$: $ S_3 \cap U = S_6 \cap U = 1$ – remis, mniejszy indeks | wybór S_3 |

Po czwartej iteracji $U = \emptyset$, pętla się kończy.

Wynik. Algorytm zachłanny zwraca pokrycie złożone z czterech zbiorów:

$$\mathcal{C} = \{S_1, S_4, S_5, S_3\}$$

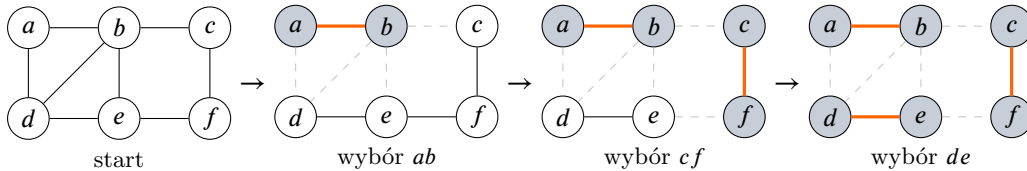
Nie jest ono optymalne: $\mathcal{C}^* = \{S_3, S_4, S_5\}$ pokrywa całe X trzema zbiorami. Pierwszy, najobfitszy wybór S_1 okazał się pułapką — po nim do domknięcia pokrycia trzeba było aż trzech kolejnych zbiorów.

Approx Vertex Cover – instancja 1

Algorytm AVC startuje z $F = E(G)$ i dopóki zostają niepokryte krawędzie, wybiera dowolną krawędź $uv \in F$, dorzuca do pokrycia C oba jej końce, a następnie usuwa z F wszystkie krawędzie incydentne z u lub v . Wybór krawędzi jest dowolny — przyjmijmy kolejność leksykograficzną

najmniejszej pozostałej krawędzi. Krawędzie wybrane (skojarzenie B) zaznaczono na **pomarańczowo**, krawędzie usuwane z F wraz z nimi — szaro, a wierzchołki dokładane do C — na niebiesko.
Dygresja. Przebieg pętli (F – niepokryte krawędzie, B – wybierane krawędzie):

1. $F = \{ab, ad, bc, bd, be, cf, de, ef\}$. Wybór ab ; usuwamy ab, ad, bc, bd, be (incydentne z a lub b).
2. $F = \{cf, de, ef\}$. Wybór cf ; usuwamy cf, ef (incydentne z f).
3. $F = \{de\}$. Wybór de ; usuwamy de — $F = \emptyset$.



Wynik. Algorytm dokłada oba końce trzech wybranych krawędzi $B = \{ab, cf, de\}$, zwracając pokrycie

$$C = \{a, b, c, d, e, f\}$$

o liczności $|C| = 2|B| = 6$ — czyli wszystkie wierzchołki grafu. Optymalne pokrycie ma rozmiar 3, np. $C^* = \{b, d, f\}$ (każda krawędź ma koniec w b, d lub f). Ta instancja realizuje więc najgorszy przypadek: stosunek $|C|/|C^*| = 2$, dokładnie gwarancja aproksymacji AVC.

Exact Subset Sum – instancja 1

Dane: $S = \{1, 4, 5, 11, 12\}$ (w kolejności $x_1, \dots, x_5$) oraz $t = 21$. Algorytm EXACTSUBSETSUM startuje z $L_0 = (0)$ i w i -tej iteracji scala L_{i-1} z $L_{i-1} + x_i$ (procedura MERGELISTS – posortowane złączenie bez duplikatów), a następnie usuwa z wyniku wszystkie liczby $> t$. Lista L_i to posortowane sumy podzbiorów $\{x_1, \dots, x_i\}$ nieprzekraczające t . Element wprowadzony przez przesunięcie $L_{i-1} + x_i$ wyróżniono na **pomarańczowo**.

Dygresja. Przebieg iteracji ($L_{i-1} + x_i$ to składnik przesunięty):

1. $x_1 = 1$: $(0) \cup (1) = (0, 1)$ $L_1 = (0, 1)$
2. $x_2 = 4$: $(0, 1) \cup (4, 5) = (0, 1, 4, 5)$ $L_2 = (0, 1, 4, 5)$
3. $x_3 = 5$: $(0, 1, 4, 5) \cup (6, 9, 10)$
(przesunięte 5, 6, 9, 10; 5 już było) $L_3 = (0, 1, 4, 5, 6, 9, 10)$
4. $x_4 = 11$: scalenie z $(11, 12, 15, 16, 17, 20, 21)$, nic > 21
 $L_4 = (0, 1, 4, 5, 6, 9, 10, 11, 12, 15, 16, 17, 20, 21)$
5. $x_5 = 12$: przesunięcie $L_4 + 12$ daje 12, 13, 16, 17, 18, 21, 22, 23, 24, 27, 28, 29, 32, 33; po usunięciu > 21 dochodzą tylko 13 i 18
 $L_5 = (0, 1, 4, 5, 6, 9, 10, 11, 12, 13, 15, 16, 17, 18, 20, 21)$

Wynik. Największy element listy L_5 nieprzekraczający $t = 21$ to

$$z = 21$$

czyli dokładnie t – istnieje podzbiór o sumie równej progowi, np. $\{4, 5, 12\}$ lub $\{1, 4, 5, 11\}$. Wersja decyzyjna ma więc odpowiedź *tak*, a optymalna suma $\leq t$ wynosi 21.

Approx Subset Sum – instancja 1

Dane: $S = \{106, 103, 205, 104\}$ (w kolejności $x_1, \dots, x_4$), $t = 312$ oraz $\varepsilon = 0,4$. Wtedy $n = 4$ i próg przycinania to $\delta = \frac{\varepsilon}{2n} = \frac{0,4}{8} = 0,05$ (mnożnik $1 + \delta = 1,05$). Startujemy z $L_0 = (0)$.

W każdej iteracji najpierw scalamy L_{i-1} z $L_{i-1} + x_i$, potem usuwamy elementy $> t = 312$, a na końcu przycinamy. Przy przycinaniu zachowujemy element y , gdy $y > last \cdot 1,05$, gdzie $last$ to ostatnio zachowany element.

Dygresja.

- $i = 1, x_1 = 106$.
 scalenie: $(0) \cup (106) = (0, 106)$;
 po usunięciu > 312 : $(0, 106)$;
 przycinanie:

0		zachowane, $last = 0$
106	$> 0 \cdot 1,05 = 0$	zachowane, $last = 106$

 $L_1 = (0, 106)$.
- $i = 2, x_2 = 103$.
 scalenie: $(0, 106) \cup (103, 209) = (0, 103, 106, 209)$;
 po usunięciu > 312 : bez zmian;
 przycinanie:

0		zachowane, $last = 0$
103	> 0	zachowane, $last = 103$
106	$\leq 103 \cdot 1,05 = 108,15$	usunięte
209	$> 108,15$	zachowane, $last = 209$

 $L_2 = (0, 103, 209)$.
- $i = 3, x_3 = 205$.
 scalenie: $(0, 103, 209) \cup (205, 308, 414) = (0, 103, 205, 209, 308, 414)$;
 po usunięciu > 312 : $(0, 103, 205, 209, 308)$;
 przycinanie:

0		zachowane, $last = 0$
103	> 0	zachowane, $last = 103$
205	$> 103 \cdot 1,05 = 108,15$	zachowane, $last = 205$
209	$\leq 205 \cdot 1,05 = 215,25$	usunięte
308	$> 215,25$	zachowane, $last = 308$

 $L_3 = (0, 103, 205, 308)$.
- $i = 4, x_4 = 104$.
 scalenie: $(0, 103, 205, 308) \cup (104, 207, 309, 412) = (0, 103, 104, 205, 207, 308, 309, 412)$;
 po usunięciu > 312 : $(0, 103, 104, 205, 207, 308, 309)$;
 przycinanie:

0		zachowane, $last = 0$
103	> 0	zachowane, $last = 103$
104	$\leq 103 \cdot 1,05 = 108,15$	usunięte
205	$> 108,15$	zachowane, $last = 205$
207	$\leq 205 \cdot 1,05 = 215,25$	usunięte
308	$> 215,25$	zachowane, $last = 308$
309	$\leq 308 \cdot 1,05 = 323,4$	usunięte

 $L_4 = (0, 103, 205, 308)$.

Wynik. Zwracamy największy element listy L_4 :

$$\boxed{z = 308}$$

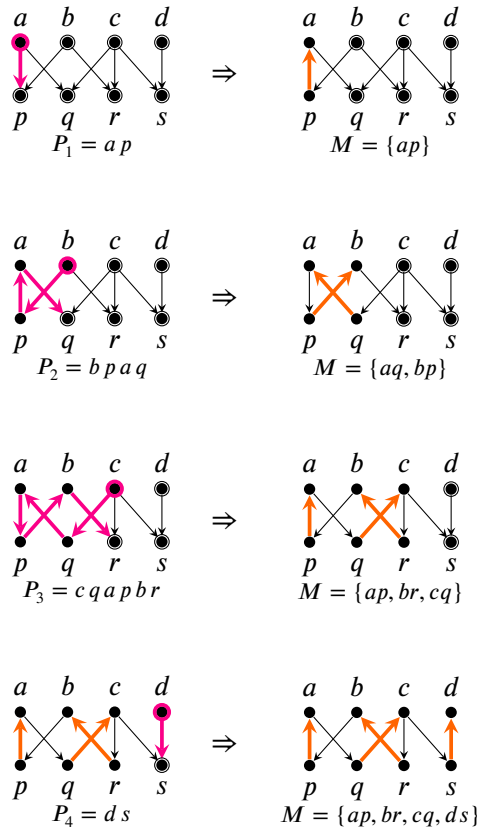
realizowany podzbiorem $\{103, 205\}$. Optimum w granicy progu to $z^* = 311$ (podzbiór $\{106, 205\}$): zostało utracone już w iteracji $i = 2$, gdy przycinanie usunęło 106 jako bliskie zachowanemu 103. Iloraz $\frac{z^*}{z} = \frac{311}{308} \approx 1,010 \leq 1 + \varepsilon = 1,4$ – zgodnie z gwarancją aproksymacji.

Maximum Matching – instancja 1

Graf dwudzielny $G = (V_1, V_2; E)$ z $V_1 = \{a, b, c, d\}$, $V_2 = \{p, q, r, s\}$ oraz krawędziami $ap, aq, bp, br, cq, cr, cs, ds$. Algorytm MM startuje z $M = \emptyset$ i dopóki APS zwraca ścieżkę powiększającą P , podmienia $M \leftarrow M \div E(P)$ (różnica symetryczna – krawędzie wolne na P wchodzą do M , skojarzone wychodzą). APS szuka ścieżki w grafie skierowanym G_M (krawędzie wolne kierowane $V_1 \rightarrow V_2$, skojarzone $V_2 \rightarrow V_1$) z wierzchołka wolnego w V_1 do wolnego w V_2 ; przyjmujemy przeszukiwanie w głąb, odwiedzając sąsiadów alfabetycznie. Na rysunkach pierścień otacza wierzchołki *wolne*, a strzałki pokazują orientację krawędzi w G_M (skojarzone w górę, do V_1 ; wolne w dół, do V_2). Każdy wiersz pokazuje G_M ze znalezioną ścieżką powiększającą (na różowo, różowy pierścień to wierzchołek startowy), a po strzałce $\Rightarrow$ – wynikowe skojarzenie M (na pomarańczowo).

Dygresja. Przebieg pętli (ścieżki powiększające na różowo, krawędzie w skojarzeniu na pomarańczowo):

1. $M = \emptyset$, wszystkie wierzchołki wolne. APS z a znajduje wolną krawędź $P_1 = ap$. $M = \{ap\}$
2. Wolne: b, c, d oraz q, r, s . W G_M z b : krawędź wolna $b \rightarrow p$, dalej skojarzona $p \rightarrow a$, wolna $a \rightarrow q$ (q wolne). Ścieżka powiększająca $P_2 = b p a q$ (zamiana: ap wychodzi, bp, aq wchodzą). $M = \{aq, bp\}$
3. Wolne: c, d oraz r, s . Z c : $c \rightarrow q$, $q \rightarrow a$, $a \rightarrow p$, $p \rightarrow b$, $b \rightarrow r$ (r wolne). Ścieżka $P_3 = c q a p b r$ (aq, bp wychodzą, cq, ap, br wchodzą). $M = \{ap, br, cq\}$
4. Wolne: d oraz s . Jedyna krawędź d to wolna ds : $P_4 = ds$. $M = \{ap, br, cq, ds\}$
5. Brak wolnych wierzchołków w V_1 – APS zwraca NULL, pętla się kończy.



Wynik. Algorytm zwraca skojarzenie

$$M = \{ap, br, cq, ds\}$$

o liczności $|M| = 4$. Jest ono *doskonale* – każdy wierzchołek jest skojarzony – więc i najliczniejsze (większe niż $|V_1| = 4$ być nie może). Dwie środkowe iteracje pokazują istotę metody: zachłanne krawędzie ap , a potem aq, bp blokowałyby kolejne wierzchołki, ale ścieżki powiększające P_2 i P_3 przekładają skojarzenie wzdłuż naprzemiennych krawędzi i za każdym razem zwiększają $|M|$ o 1.

Para najbliższych punktów – instancja 2

Dane (instancja podana tabelą), $n = 8$. Ponumerujemy punkty w kolejności wejściowej:

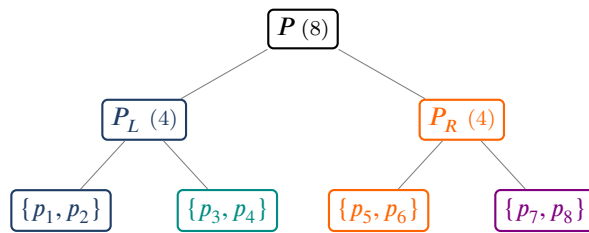
$$p_1 = (0, 0), p_2 = (1, 6), p_3 = (4, 2), p_4 = (5, 5), p_5 = (7, 1), p_6 = (8, 7), p_7 = (9, 3), p_8 = (11, 6).$$

Sortowanie wstępne. Tablice X (rosnąco po x) i Y (rosnąco po y):

$$X = [p_1(0, 0), p_2(1, 6), p_3(4, 2), p_4(5, 5), p_5(7, 1), p_6(8, 7), p_7(9, 3), p_8(11, 6)],$$

$$Y = [p_1(0, 0), p_5(7, 1), p_3(4, 2), p_7(9, 3), p_4(5, 5), p_2(1, 6), p_8(11, 6), p_6(8, 7)].$$

Dziel. $|P| = 8 > 3$, dzielimy prostą l : $x = 6$ (mediana współrzędnej x : $\frac{X(4).x + X(5).x}{2} = \frac{5+7}{2} = 6$; zob. uwagę o wyborze l) na $|P_L| = |P_R| = 4$, a każdą połowę dalej, aż do podzbiorów po 2 punkty (liście — sprawdzane siłowo). Strukturę rekursji zbiera rysunek 3.



Rysunek 3: Drzewo rekursji. Cztery liście (po 2 punkty) liczone są siłowo; ich kolory powtarzają się na rysunku 4.

$$P_L = \{p_1, p_2, p_3, p_4\}, \quad P_R = \{p_5, p_6, p_7, p_8\}.$$

Zwycięzaj — lewa połowa P_L . Liście (po 2 punkty — siłowo) i faza *połącz* wzdłuż prostej $x = 2,5$:

$$d(p_1, p_2) = \sqrt{1^2 + 6^2} = \sqrt{37} \approx 6,08, \quad d(p_3, p_4) = \sqrt{1^2 + 3^2} = \sqrt{10} \approx 3,16.$$

Pas o szerokości $2\sqrt{10}$ obejmuje wszystkie cztery punkty, ale żadna para „międzystronowa” nie jest bliższa niż $\sqrt{10}$. Zatem $\delta_L = \sqrt{10}$ dla pary $(p_3(4, 2), p_4(5, 5))$.

Zwycięzaj — prawa połowa P_R . Liście i faza *połącz* wzdłuż $x = 8,5$:

$$d(p_5, p_6) = \sqrt{1^2 + 6^2} = \sqrt{37} \approx 6,08, \quad d(p_7, p_8) = \sqrt{2^2 + 3^2} = \sqrt{13} \approx 3,61.$$

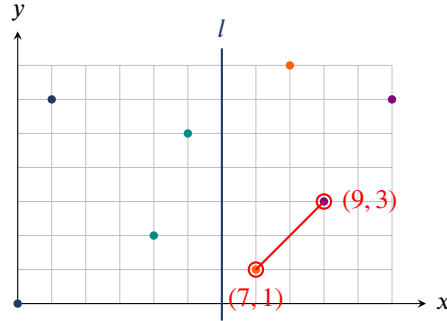
W pasie wokół $x = 8,5$ leżą p_5, p_6, p_7, p_8 ; sprawdzając pary stwierdzamy

$$d(p_5, p_7) = \sqrt{2^2 + 2^2} = \sqrt{8} = 2\sqrt{2} \approx 2,83,$$

co poprawia δ . Zatem $\delta_R = \sqrt{8}$ dla pary $(p_5(7, 1), p_7(9, 3))$.

Połącz — poziom górny. $\delta = \min(\delta_L, \delta_R) = \min(\sqrt{10}, \sqrt{8}) = \sqrt{8}$. Pas wokół $l: x = 6$ ma szerokość $2\sqrt{8} \approx 5,66$: $x \in [6 - 2\sqrt{2}, 6 + 2\sqrt{2}] \approx [3,17, 8,83]$. W pasie (rosnąco po y) leżą $p_5(7, 1), p_3(4, 2), p_4(5, 5), p_6(8, 7)$; sąsiednie pary mają odległości $\geq \sqrt{10} > \sqrt{8}$, więc δ bez zmian.

Wynik. Para najbliższych punktów to $(7, 1)$ i $(9, 3)$ w odległości $\delta = 2\sqrt{2} \approx 2,83$.



Rysunek 4: Instancja 2: prosta podziału $l: x = 6$, punkty pokolorowane wg liścia drzewa rekursji oraz znaleziona para $(7, 1), (9, 3)$ (czerwona).

Matrix-Chain-Order – instancja 2

Dane: $n = 5$, $p = (10, 3, 12, 5, 50, 6)$, czyli

$$A_1 (10 \times 3), \quad A_2 (3 \times 12), \quad A_3 (12 \times 5), \quad A_4 (5 \times 50), \quad A_5 (50 \times 6).$$

Wypełniamy tablice m i s przekątnymi po rosnącej długości podciągu L ; w każdym kroku dopisane komórki wyróżniono na pomarańczowo, a $\times$ to pola nieużywane. Skrót: $A_i = p_{i-1} \times p_i$.

Stan początkowy ($L = 1$, koszt 0):

$$m \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ 1 & 0 & & & & \\ 2 & \times & 0 & & & \\ 3 & \times & \times & 0 & & \\ 4 & \times & \times & \times & 0 & \\ 5 & \times & \times & \times & \times & 0 \end{bmatrix}$$

$$s \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ 1 & \times & & & & \\ 2 & \times & \times & & & \\ 3 & \times & \times & \times & & \\ 4 & \times & \times & \times & \times & \\ 5 & \times & \times & \times & \times & \times \end{bmatrix}$$

i	0	1	2	3	4	5
p_i	10	3	12	5	50	6

 $A_i = p_{i-1} \times p_i$

$$\begin{aligned} L = 2 : \quad & m(1, 2) = 10 \cdot 3 \cdot 12 = 360 \\ & m(2, 3) = 3 \cdot 12 \cdot 5 = 180 \\ & m(3, 4) = 12 \cdot 5 \cdot 50 = 3000 \\ & m(4, 5) = 5 \cdot 50 \cdot 6 = 1500 \end{aligned}$$

po $L = 2$:

$$m \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ 1 & 0 & 360 & & & \\ 2 & \times & 0 & 180 & & \\ 3 & \times & \times & 0 & 3000 & \\ 4 & \times & \times & \times & 0 & 1500 \\ 5 & \times & \times & \times & \times & 0 \end{bmatrix}$$

$$s \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ 1 & \times & 1 & & & \\ 2 & \times & \times & 2 & & \\ 3 & \times & \times & \times & 3 & \\ 4 & \times & \times & \times & \times & 4 \\ 5 & \times & \times & \times & \times & \times \end{bmatrix}$$

i	0	1	2	3	4	5
p_i	10	3	12	5	50	6

 $A_i = p_{i-1} \times p_i$

$$\begin{aligned}
L = 3 : \quad m(1,3) &= \min \begin{cases} m(1,1) + m(2,3) + 10 \cdot 3 \cdot 5 \\ m(1,2) + m(3,3) + 10 \cdot 12 \cdot 5 \end{cases} = \min \begin{cases} 0 + 180 + 150 \\ 360 + 0 + 600 \end{cases} = \min \begin{cases} 330 \\ 960 \end{cases} = 330 \\
m(2,4) &= \min \begin{cases} m(2,2) + m(3,4) + 3 \cdot 12 \cdot 50 \\ m(2,3) + m(4,4) + 3 \cdot 5 \cdot 50 \end{cases} = \min \begin{cases} 0 + 3000 + 1800 \\ 180 + 0 + 750 \end{cases} = \min \begin{cases} 4800 \\ 930 \end{cases} = 930 \\
m(3,5) &= \min \begin{cases} m(3,3) + m(4,5) + 12 \cdot 5 \cdot 6 \\ m(3,4) + m(5,5) + 12 \cdot 50 \cdot 6 \end{cases} = \min \begin{cases} 0 + 1500 + 360 \\ 3000 + 0 + 3600 \end{cases} = \min \begin{cases} 1860 \\ 6600 \end{cases} = 1860
\end{aligned}$$

po $L = 3$:

m	$i \backslash j$	1	2	3	4	5
	1	0	360	330		
	2	×	0	180	930	
	3	×	×	0	3000	1860
	4	×	×	×	0	1500
	5	×	×	×	×	0

s	$i \backslash j$	1	2	3	4	5
	1	×	1	1		
	2	×	×	2	3	
	3	×	×	×	3	3
	4	×	×	×	×	4
	5	×	×	×	×	×

i	0	1	2	3	4	5
p_i	10	3	12	5	50	6

 $A_i = p_{i-1} \times p_i$

$$\begin{aligned}
L = 4 : \quad m(1,4) &= \min \begin{cases} m(1,1) + m(2,4) + 10 \cdot 3 \cdot 50 \\ m(1,2) + m(3,4) + 10 \cdot 12 \cdot 50 \\ m(1,3) + m(4,4) + 10 \cdot 5 \cdot 50 \end{cases} = \min \begin{cases} 0 + 930 + 1500 \\ 360 + 3000 + 6000 \\ 330 + 0 + 2500 \end{cases} = \min \begin{cases} 2430 \\ 9360 \\ 2830 \end{cases} = 2430 \\
m(2,5) &= \min \begin{cases} m(2,2) + m(3,5) + 3 \cdot 12 \cdot 6 \\ m(2,3) + m(4,5) + 3 \cdot 5 \cdot 6 \\ m(2,4) + m(5,5) + 3 \cdot 50 \cdot 6 \end{cases} = \min \begin{cases} 0 + 1860 + 216 \\ 180 + 1500 + 90 \\ 930 + 0 + 900 \end{cases} = \min \begin{cases} 2076 \\ 1770 \\ 1830 \end{cases} = 1770
\end{aligned}$$

po $L = 4$:

m	$i \backslash j$	1	2	3	4	5
	1	0	360	330	2430	
	2	×	0	180	930	1770
	3	×	×	0	3000	1860
	4	×	×	×	0	1500
	5	×	×	×	×	0

s	$i \backslash j$	1	2	3	4	5
	1	×	1	1	1	
	2	×	×	2	3	3
	3	×	×	×	3	3
	4	×	×	×	×	4
	5	×	×	×	×	×

i	0	1	2	3	4	5
p_i	10	3	12	5	50	6

 $A_i = p_{i-1} \times p_i$

$$L = 5 : \quad m(1,5) = \min \begin{cases} m(1,1) + m(2,5) + 10 \cdot 3 \cdot 6 \\ m(1,2) + m(3,5) + 10 \cdot 12 \cdot 6 \\ m(1,3) + m(4,5) + 10 \cdot 5 \cdot 6 \\ m(1,4) + m(5,5) + 10 \cdot 50 \cdot 6 \end{cases} = \min \begin{cases} 0 + 1770 + 180 \\ 360 + 1860 + 720 \\ 330 + 1500 + 300 \\ 2430 + 0 + 3000 \end{cases} = \min \begin{cases} 1950 \\ 2940 \\ 2130 \\ 5430 \end{cases} = 1950$$

tablice końcowe:

m	$i \backslash j$	1	2	3	4	5
	1	0	360	330	2430	1950
	2	×	0	180	930	1770
	3	×	×	0	3000	1860
	4	×	×	×	0	1500
	5	×	×	×	×	0

s	$i \backslash j$	1	2	3	4	5
	1	×	1	1	1	1
	2	×	×	2	3	3
	3	×	×	×	3	3
	4	×	×	×	×	4
	5	×	×	×	×	×

Optymalne nawiasowanie. Odtwarzamy je z tablicy s :

$$s(1, 5) = 1 \Rightarrow (A_1)(A_2 A_3 A_4 A_5), \quad s(2, 5) = 3 \Rightarrow (A_2 A_3)(A_4 A_5), \quad s(2, 3) = 2, \quad s(4, 5) = 4.$$

Stąd optymalne nawiasowanie

$$A_1 ((A_2 A_3)(A_4 A_5))$$

o minimalnym koszcie $m(1, 5) = 1950$ mnożeń skalarnych.

Najdłuższy wspólny podciąg (LCS) – instancja 2

Dane: $X = (G, A, T, T, A, C, A)$, $Y = (G, C, A, T, A, A, T)$, czyli $m = 7$, $n = 7$. Tablicę kosztów c wypełniamy wierszami regułą jak w instancji 1, a równolegle w tablicy b zapisujemy kierunek ($\nwarrow$ – dopasowanie, $\uparrow$ – góra, $\leftarrow$ – lewo; remisy na korzyść góry). Komórki wchodzące do wybranego LCS wyróżniono na **pomarańczowo**.

Tablica kosztów c :

	j	0	1	2	3	4	5	6	7
i	y_j	G	C	A	T	A	A	T	
0	x_i	0	0	0	0	0	0	0	0
1	G	0	1	1	1	1	1	1	1
2	A	0	1	1	2	2	2	2	2
3	T	0	1	1	2	3	3	3	3
4	T	0	1	1	2	3	3	3	4
5	A	0	1	1	2	3	4	4	4
6	C	0	1	2	2	3	4	4	4
7	A	0	1	2	3	3	4	5	5

Tablica strzałek b :

	j	0	1	2	3	4	5	6	7
i	y_j	G	C	A	T	A	A	T	
0	x_i								
1	G		$\nwarrow$	$\leftarrow$	$\leftarrow$	$\leftarrow$	$\leftarrow$	$\leftarrow$	$\leftarrow$
2	A		$\uparrow$	$\uparrow$	$\nwarrow$	$\leftarrow$	$\nwarrow$	$\nwarrow$	$\leftarrow$
3	T		$\uparrow$	$\uparrow$	$\uparrow$	$\nwarrow$	$\leftarrow$	$\leftarrow$	$\nwarrow$
4	T		$\uparrow$	$\uparrow$	$\uparrow$	$\nwarrow$	$\uparrow$	$\uparrow$	$\nwarrow$
5	A		$\uparrow$	$\uparrow$	$\nwarrow$	$\uparrow$	$\nwarrow$	$\nwarrow$	$\uparrow$
6	C		$\uparrow$	$\nwarrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$
7	A		$\uparrow$	$\uparrow$	$\nwarrow$	$\uparrow$	$\nwarrow$	$\nwarrow$	$\leftarrow$

Dygresja. Po jednej komórce na każdą gałąź reguły, $X = (x_1, \dots, x_7)$, $Y = (y_1, \dots, y_7)$:

- **Dopasowanie** ($\nwarrow$): $c(1, 1)$, $x_1 = G = y_1$, więc $c(1, 1) = c(0, 0) + 1 = 1$.
- **Góra** ($\uparrow$): $c(2, 1)$, $x_2 = A \neq G = y_1$; sąsiedzi $c(1, 1) = 1 \geq c(2, 0) = 0$, więc $c(2, 1) = 1$.
- **Lewo** ($\leftarrow$): $c(1, 2)$, $x_1 = G \neq C = y_2$; sąsiedzi $c(0, 2) = 0 < c(1, 1) = 1$, więc $c(1, 2) = 1$.
- **Remis**: $c(2, 7)$, $A \neq T$, $c(1, 7) = c(2, 6) = 2$ – warunek $\geq$ spełniony, więc $\uparrow$.

Odtworzenie podciągu. Startujemy z $(m, n) = (7, 7)$ i idziemy za strzałkami; przy $\nwarrow$ schodzimy po przekątnej, dopisując x_i do LCS:

$$(7, 7) \xrightarrow{\nwarrow} (7, 6) \xrightarrow{\nwarrow} (6, 5) [x_7 = A] \xrightarrow{\uparrow} (5, 5) \xrightarrow{\nwarrow} (4, 4) [x_5 = A] \\ \xrightarrow{\nwarrow} (3, 3) [x_4 = T] \xrightarrow{\uparrow} (2, 3) \xrightarrow{\nwarrow} (1, 2) [x_2 = A] \xrightarrow{\nwarrow} (1, 1) \xrightarrow{\nwarrow} (0, 0) [x_1 = G].$$

Znaki dopisywane są *po* powrocie z rekurencji, więc w kolejności od najgłębszego: G, A, T, A, A . Stąd najdłuższy wspólny podciąg

$$\text{LCS}(X, Y) = (G, A, T, A, A)$$

o długości $c(7, 7) = 5$.

Zachłanny wybór zajęć – instancja 2

Dane ($n = 9$ zajęć). Tym razem nie są one posortowane po czasie zakończenia: $t_2 = 5 > t_3 = 4$. Najpierw więc sortujemy rosnąco po t_i (miejscami zamieniają się z_2 i z_3 oraz z_4 i z_5):

i	1	3	2	5	4	6	7	8	9
s_i	0	1	2	4	6	8	7	9	11
t_i	3	4	5	7	8	9	11	12	14

Algorytm GREEDYACTIVITYSELECTOR wybiera pierwsze zajęcie i idzie dalej, dodając z_i wtedy i tylko wtedy, gdy $s_i \geq t_j$ dla ostatnio wybranego z_j . Wybrane zajęcia wyróżniono na **pomarańczowo**.

Dygresja. Przebieg pętli (kolejność po sortowaniu: $z_1, z_3, z_2, z_5, z_4, z_6, z_7, z_8, z_9$; początkowo $A = \{z_1\}$, $j = 1$, $t_j = 3$):

- z_3 : $s_3 = 1 \geq t_j = 3$? *nie* – pomijamy.
- z_2 : $s_2 = 2 \geq 3$? *nie* – pomijamy.
- z_5 : $s_5 = 4 \geq 3$? *tak* – $A \cup \{z_5\}$, $j \leftarrow 5$, $t_j = 7$.
- z_4 : $s_4 = 6 \geq 7$? *nie* – pomijamy.
- z_6 : $s_6 = 8 \geq 7$? *tak* – $A \cup \{z_6\}$, $j \leftarrow 6$, $t_j = 9$.
- z_7 : $s_7 = 7 \geq 9$? *nie* – pomijamy.
- z_8 : $s_8 = 9 \geq 9$? *tak* – $A \cup \{z_8\}$, $j \leftarrow 8$, $t_j = 12$.
- z_9 : $s_9 = 11 \geq 12$? *nie* – pomijamy.

Wynik. Wybrane zajęcia są parami zgodne, bo ich przedziały $\langle 0, 3 \rangle$, $\langle 4, 7 \rangle$, $\langle 8, 9 \rangle$, $\langle 9, 12 \rangle$ są rozłączne:

$$A = \{z_1, z_5, z_6, z_8\}$$

o liczności $|A| = 4$.

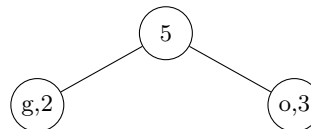
Algorytm Huffmana – instancja 2

Dane: kolejka startowa $(g, 2)$, $(o, 3)$, $(k, 6)$, $(m, 8)$, $(l, 11)$, $(u, 14)$, $(e, 20)$, sumaryczna częstość 64. W każdej z $n - 1 = 6$ iteracji łączymy dwa korzenie o *najmniejszej* częstości (mniejszy jako lewe dziecko, krawędź 0; remisy wg kolejności w kolejce), wstawiając nowy wierzchołek z powrotem. Style drzew i makro \huffstep przejęte z instancji 1.

1. Łączymy g i o .

Zostają:

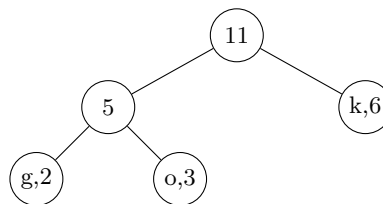
(5) , $(k, 6)$, $(m, 8)$, $(l, 11)$, $(u, 14)$, $(e, 20)$.



2. Łączymy 5 i k.

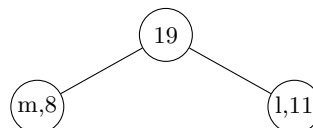
Zostają:

$(m, 8), (11), (l, 11), (u, 14), (e, 20)$.



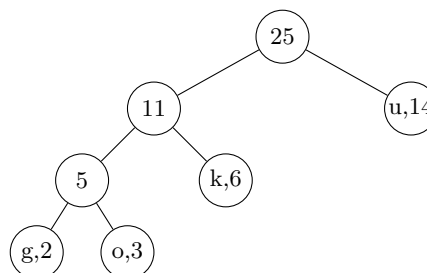
3. Łączymy m i l.

Zostają: $(11), (u, 14), (19), (e, 20)$.



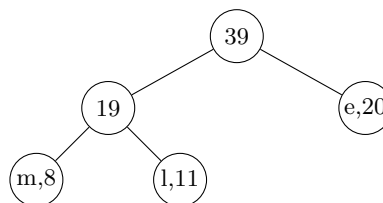
4. Łączymy 11 i u.

Zostają: $(19), (e, 20), (25)$.



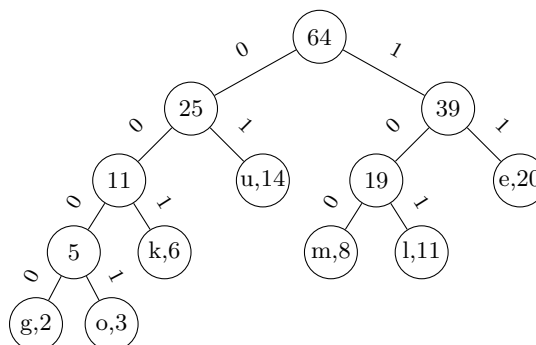
5. Łączymy 19 i e.

Zostają: $(25), (39)$.



6. Łączymy 25 i 39.

(ostateczne drzewo kodów)



Odczytany kod z drzewa w kroku 6:

$u \Rightarrow 01$

$e \Rightarrow 11$

$k \Rightarrow 001$

$m \Rightarrow 100$

$l \Rightarrow 101$

$g \Rightarrow 0000$

$o \Rightarrow 0001$

Liczba bitów.

$$B(T) = 2 \cdot 4 + 3 \cdot 4 + 6 \cdot 3 + 8 \cdot 3 + 11 \cdot 3 + 14 \cdot 2 + 20 \cdot 2 = 163 \text{ bitów}$$

(wobec $64 \cdot 3 = 192$ bitów przy kodzie stałej długości po 3 bity na znak).

Szeregowanie zadań z deadlineami – instancja 2

Dane: $n = 7$ zadań o jednostkowym czasie wykonania. Kary w_i są już ułożone nierosnąco ($w_1 \geq \dots \geq w_7$), więc zadania rozważamy w kolejności $z_1, \dots, z_7$. Algorytm GREEDY dodaje z_i do A wtedy i tylko wtedy, gdy zbiór pozostaje wolny od kar, tzn. profil $N_t(A) = |\{z_j \in A : d_j \leq t\}|$ nie przekracza $(1, 2, \dots, n)$. Zadania terminowe wyróżniono na **pomarańczowo**.

i	1	2	3	4	5	6	7
d_i	1	1	3	2	5	4	3
w_i	60	55	45	35	25	20	10

Dygresja. Poniżej A wypisane rosnąco po deadlineach, a po prawej profil $N(A) = (N_1, \dots, N_7)$, który musi pozostać $\leq (1, 2, 3, 4, 5, 6, 7)$:

- | | |
|---|--------------------------------|
| 1. $A = (z_1)$ | $N(A) = (1, 1, 1, 1, 1, 1, 1)$ |
| 2. z_2 ($d = 1$) nie dodajemy, bo $N_1(A \cup \{z_2\}) = 2 > 1$. | |
| 3. $A = (z_1, z_3)$ | $N(A) = (1, 1, 2, 2, 2, 2, 2)$ |
| 4. $A = (z_1, z_4, z_3)$ | $N(A) = (1, 2, 3, 3, 3, 3, 3)$ |
| 5. $A = (z_1, z_4, z_3, z_5)$ | $N(A) = (1, 2, 3, 3, 4, 4, 4)$ |
| 6. $A = (z_1, z_4, z_3, z_6, z_5)$ | $N(A) = (1, 2, 3, 4, 5, 5, 5)$ |
| 7. z_7 ($d = 3$) nie dodajemy, bo $N_3(A \cup \{z_7\}) = 4 > 3$. | |

Harmonogram. Zadania terminowe ustawiamy rosnąco po deadlineach, a spóźnione (z_2, z_7) na końcu:

$$(z_1, z_4, z_3, z_6, z_5, z_2, z_7)$$

Suma kar zadań spóźnionych: $w_2 + w_7 = 55 + 10 = 65$.

Greedy Set Cover – instancja 2

Dane: $X = \{1, \dots, 9\}$ oraz $\mathcal{F} = \{S_1, \dots, S_5\}$. Algorytm GSC w każdej iteracji dokłada do $\mathcal{C}$ zbiór maksymalizujący $|S \cap U|$ (remisy na korzyść mniejszego indeksu) i odejmuje $U \leftarrow U \setminus S$. Zbiory wybrane wyróżniono na **pomarańczowo**.

S	elementy	$ S $
S_1	$\{1, 2, 3, 4, 5\}$	5
S_2	$\{6, 7, 8, 9\}$	4
S_3	$\{1, 4, 7\}$	3
S_4	$\{2, 5, 8\}$	3
S_5	$\{3, 6, 9\}$	3

Dygresja. W każdym wierszu po lewej bieżący zbiór niepokrytych U , a po prawej rozważane przecięcia $|S \cap U|$ (pogrubiony zwycięzca):

1. $U = \{1, \dots, 9\}$: $|S_1 \cap U| = 5$, $|S_2 \cap U| = 4$, $|S_3 \cap U| = |S_4 \cap U| = |S_5 \cap U| = 3$ wybór S_1

$$2. U = \{6, 7, 8, 9\}: |S_2 \cap U| = 4, |S_5 \cap U| = 2, |S_3 \cap U| = |S_4 \cap U| = 1 \quad \text{wybór } S_2$$

Po drugiej iteracji $U = \emptyset$, pętla się kończy.

Wynik. Algorytm zachłanny zwraca pokrycie

$$\mathcal{C} = \{S_1, S_2\}$$

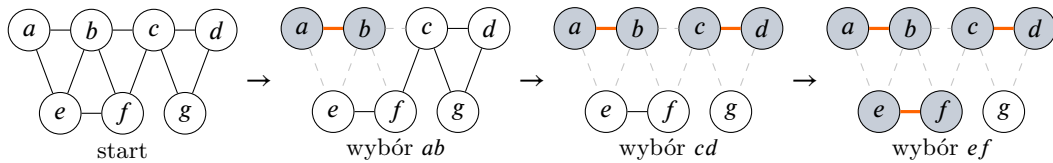
o rozmiarze 2. Ponieważ $S_1 \cup S_2 = X$, a żaden pojedynczy zbiór nie pokrywa całego X , jest to zarazem pokrycie optymalne ($|\mathcal{C}^*| = 2$) — tutaj zachłanność trafia w optimum.

Approx Vertex Cover – instancja 2

Algorytm AVC dopóki zostają niepokryte krawędzie, wybiera (tu: leksykograficznie najmniejszą) krawędź $uv \in F$, dorzuca do pokrycia C oba jej końce i usuwa z F wszystkie krawędzie incydentne z u lub v . Krawędzie wybrane (skojarzenie B) na **pomarańczowo**, usuwane wraz z nimi — szaro, wierzchołki dokładane do C — niebiesko.

Dygresja. Przebieg pętli (F – niepokryte krawędzie, B – wybierane krawędzie):

1. $F = \{ab, ae, bc, be, bf, cd, cf, cg, dg, ef\}$. Wybór ab ; usuwamy ab, ae, bc, be, bf (incydentne z a lub b).
2. $F = \{cd, cf, cg, dg, ef\}$. Wybór cd ; usuwamy cd, cf, cg, dg (incydentne z c lub d).
3. $F = \{ef\}$. Wybór ef ; usuwamy ef — $F = \emptyset$.



Wynik. Algorytm dokłada oba końce trzech wybranych krawędzi $B = \{ab, cd, ef\}$, zwracając pokrycie

$$C = \{a, b, c, d, e, f\}$$

o liczności $|C| = 2|B| = 6$ (wierzchołek g nie wszedł). Optymalne pokrycie ma rozmiar 4, np. $C^* = \{b, c, d, e\}$: każda krawędź ma koniec w b, c, d lub e . Iloraz $|C|/|C^*| = 6/4 = 1,5 \leq 2$, zgodnie z gwarancją aproksymacji AVC.

Exact Subset Sum – instancja 2

Dane: $S = \{2, 3, 7, 8, 10\}$ (w kolejności $x_1, \dots, x_5$) oraz $t = 15$. Algorytm EXACTSUBSETSUM startuje z $L_0 = (0)$ i w i -tej iteracji scala L_{i-1} z $L_{i-1} + x_i$, po czym usuwa elementy $> t$. Element wprowadzony przez przesunięcie $L_{i-1} + x_i$ wyróżniono na **pomarańczowo**.

Dygresja. Przebieg iteracji ($L_{i-1} + x_i$ to składnik przesunięty):

1. $x_1 = 2$: $(0) \cup (2) = (0, 2)$ $L_1 = (0, 2)$
2. $x_2 = 3$: $(0, 2) \cup (3, 5) = (0, 2, 3, 5)$ $L_2 = (0, 2, 3, 5)$
3. $x_3 = 7$: scalenie z $(7, 9, 10, 12)$, nic > 15
 $L_3 = (0, 2, 3, 5, 7, 9, 10, 12)$
4. $x_4 = 8$: przesunięcie $L_3 + 8$ daje $8, 10, 11, 13, 15, 17, 18, 20$; po usunięciu > 15 dochodzą **8, 11, 13, 15** (10 już było)
 $L_4 = (0, 2, 3, 5, 7, 8, 9, 10, 11, 12, 13, 15)$
5. $x_5 = 10$: przesunięcie $L_4 + 10$ daje $10, 12, 13, 15, 17, \dots$; wszystkie ≤ 15 ($10, 12, 13, 15$) już są na liście, więc *nic nowego* $L_5 = L_4$

Wynik. Największy element listy L_5 nieprzekraczający $t = 15$ to

$$z = 15$$

czyli dokładnie t – istnieje podzbiór o sumie równej progowi, np. $\{7, 8\}$ lub $\{2, 3, 10\}$. Wersja decyzyjna ma więc odpowiedź *tak*, a optymalna suma $\leq t$ wynosi 15.

Approx Subset Sum – instancja 2

Dane: $S = \{10, 12, 17, 25\}$ (w kolejności $x_1, \dots, x_4$), $t = 50$ oraz $\varepsilon = 0,5$. Wtedy $n = 4$ i próg przycinania $\delta = \frac{\varepsilon}{2n} = \frac{0,5}{8} = 0,0625$ (mnożnik $1 + \delta = 1,0625$). Startujemy z $L_0 = (0)$.

W każdej iteracji najpierw scalamy L_{i-1} z $L_{i-1} + x_i$, potem usuwamy elementy $> t = 50$, a na końcu przycinamy: zachowujemy y , gdy $y > last \cdot 1,0625$, gdzie *last* to ostatnio zachowany element.

Dygresja.

- $i = 1, x_1 = 10$.
 scalenie: $(0) \cup (10) = (0, 10)$; po usunięciu > 50 : bez zmian;
 przycinanie:
 0 zachowane, $last = 0$
 10 > 0 zachowane, $last = 10$
 $L_1 = (0, 10)$.
- $i = 2, x_2 = 12$.
 scalenie: $(0, 10) \cup (12, 22) = (0, 10, 12, 22)$; po usunięciu > 50 : bez zmian;
 przycinanie:
 0 zachowane, $last = 0$
 10 > 0 zachowane, $last = 10$
 12 > $10 \cdot 1,0625 = 10,625$ zachowane, $last = 12$
 22 > $12 \cdot 1,0625 = 12,75$ zachowane, $last = 22$
 $L_2 = (0, 10, 12, 22)$.
- $i = 3, x_3 = 17$.
 scalenie: $(0, 10, 12, 22) \cup (17, 27, 29, 39) = (0, 10, 12, 17, 22, 27, 29, 39)$; po usunięciu > 50 : bez zmian;
 przycinanie:
 0 zachowane, $last = 0$
 10 > 0 zachowane, $last = 10$
 12 > $10 \cdot 1,0625 = 10,625$ zachowane, $last = 12$
 17 > $12 \cdot 1,0625 = 12,75$ zachowane, $last = 17$
 22 > $17 \cdot 1,0625 = 18,0625$ zachowane, $last = 22$
 27 > $22 \cdot 1,0625 = 23,375$ zachowane, $last = 27$
 29 > $27 \cdot 1,0625 = 28,6875$ zachowane, $last = 29$
 39 > $29 \cdot 1,0625 = 30,8125$ zachowane, $last = 39$
 $L_3 = (0, 10, 12, 17, 22, 27, 29, 39)$.
- $i = 4, x_4 = 25$.
 scalenie
 $(0, 10, 12, 17, 22, 27, 29, 39) \cup (25, 35, 37, 42, 47, 52, 54, 64)$
 $= (0, 10, 12, 17, 22, 25, 27, 29, 35, 37, 39, 42, 47, 52, 54, 64)$;
 po usunięciu > 50 odpadają 52, 54, 64: $(0, 10, 12, 17, 22, 25, 27, 29, 35, 37, 39, 42, 47)$;
 przycinanie:

0		<i>zachowane, last = 0</i>
10	> 0	<i>zachowane, last = 10</i>
12	> 10,625	<i>zachowane, last = 12</i>
17	> 12,75	<i>zachowane, last = 17</i>
22	> $17 \cdot 1,0625 = 18,0625$	<i>zachowane, last = 22</i>
25	> $22 \cdot 1,0625 = 23,375$	<i>zachowane, last = 25</i>
27	> $25 \cdot 1,0625 = 26,5625$	<i>zachowane, last = 27</i>
29	> $27 \cdot 1,0625 = 28,6875$	<i>zachowane, last = 29</i>
35	> $29 \cdot 1,0625 = 30,8125$	<i>zachowane, last = 35</i>
37	$\leq 35 \cdot 1,0625 = 37,1875$	<i>usunięte</i>
39	> 37,1875	<i>zachowane, last = 39</i>
42	> $39 \cdot 1,0625 = 41,4375$	<i>zachowane, last = 42</i>
47	> $42 \cdot 1,0625 = 44,625$	<i>zachowane, last = 47</i>
$L_4 = (0, 10, 12, 17, 22, 25, 27, 29, 35, 39, 42, 47)$.		

Wynik. Zwracamy największy element listy L_4 :

$$z = 47$$

realizowany podzbiorem $\{10, 12, 25\}$. Tu optimum dokładne to również $z^* = 47$ (ta sama suma), więc $\frac{z^*}{z} = 1 \leq 1 + \varepsilon = 1,5$ — przycięcie usunęło jedynie 37, nie naruszając optimum.

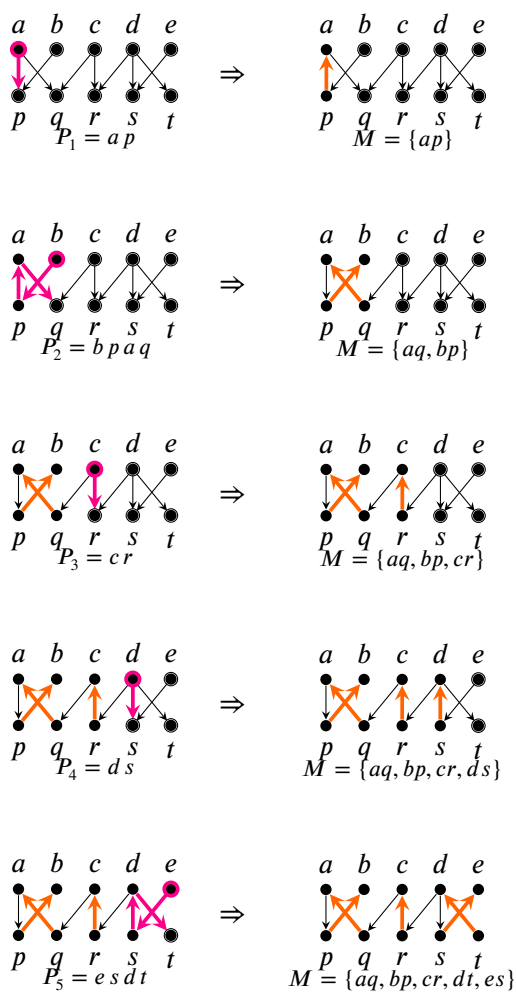
Maximum Matching – instancja 2

Graf dwudzielny $G = (V_1, V_2; E)$ z $V_1 = \{a, b, c, d, e\}$, $V_2 = \{p, q, r, s, t\}$ oraz krawędziami $ap, aq, bp, cq, cr, dr, ds, dt, es$. Algorytm MM startuje z $M = \emptyset$ i dopóki APS zwraca ścieżkę powiększającą P , podmienia $M \leftarrow M \div E(P)$. APS przeszukuje G_M w głąb (sąsiedzi alfabetycznie) z wolnego wierzchołka V_1 do wolnego w V_2 . Na rysunkach pierścień otacza wierzchołki *wolne*, a strzałki pokazują orientację krawędzi w G_M (skojarzone w górę, do V_1 ; wolne w dół, do V_2). Każdy wiersz pokazuje G_M ze znalezioną ścieżką powiększającą (na różowo), a po strzałce $\Rightarrow$ — wynikowe skojarzenie M (na pomarańczowo).

Dygresja. Przebieg pętli (ścieżki powiększające na różowo, krawędzie w skojarzeniu na pomarańczowo):

1. $M = \emptyset$. APS z a znajduje wolną krawędź $P_1 = ap$. $M = \{ap\}$
2. Z b : wolna $b \rightarrow p$, skojarzona $p \rightarrow a$, wolna $a \rightarrow q$ (q wolne). Ścieżka $P_2 = b p a q$ (ap wychodzi, bp, aq wchodzi). $M = \{aq, bp\}$
3. Z c : $c \rightarrow q$ prowadzi do martwego punktu, więc bierzemy $c \rightarrow r$ (r wolne): $P_3 = cr$. $M = \{aq, bp, cr\}$
4. Z d : $d \rightarrow r$ prowadzi do martwego punktu, więc $d \rightarrow s$ (s wolne): $P_4 = ds$. $M = \{aq, bp, cr, ds\}$
5. Z e : $e \rightarrow s$, skojarzona $s \rightarrow d$, wolna $d \rightarrow t$ (t wolne). Ścieżka $P_5 = e s d t$ (ds wychodzi, es, dt wchodzi). $M = \{aq, bp, cr, es, dt\}$

Brak wolnych wierzchołków w V_1 – APS zwraca NULL, pętla kończy się.



Wynik. Algorytm zwraca skojarzenie

$$M = \{aq, bp, cr, dt, es\}$$

o liczności $|M| = 5$. Jest ono *doskonałe* — każdy z 5 wierzchołków obu stron jest skojarzony — więc i najliczniejsze. Ścieżki powiększające P_2 i P_5 pokazują istotę metody: przekładają skojarzenie wzdłuż naprzemiennych krawędzi i zwiększają $|M|$ o 1, odblokowując kolejne wierzchołki.